

6장 임베딩 (2)

2 Word2Vec

1. 단어 벡터화
2. CBoW (Continuous Bag of Words)
3. Skip-gram
4. 계층적 소프트맥스
5. 네거티브 샘플링
6. 모델 실습: Skip-gram
7. 모델 실습: Gensim

3 fastText

1. 모델 실습

4 순환 신경망

1. 순환 신경망
2. 장단기 메모리
3. 모델 실습

5 합성곱 신경망

1. 합성곱 계층
2. 활성화 맵
3. 풀링
4. 완전 연결 계층
5. 모델 실습

2 Word2Vec

- 2013년 구글이 공개한 임베딩 모델
- 단어 간의 유사성 측정 위해 **분포 가설**을 기반으로 개발
- **분포 가설**

: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다는 가정

- 동시 발생 확률 분포 이용 → 단어 간의 유사성 측정

⇒ 단어의 분산 표현 학습 가능

- 분산 표현: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
- 분산 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남 → 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 갖게 됨

ex. '자동차' 와 '비행기' 는 벡터공간에서 서로 가까운 위치에 표현

⇒ 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미정보를 저장하지 못하는 한계 극복, 대량의 텍스트 데이터에서 단어 간의 관계 파악, 벡터 공간 상에서 유사한 단어 군집화 → 단어의 의미 정보 효과적으로 표현

- 다양한 자연어 처리 작업에서 높은 성능, 다운스트림 작업에서 더 뛰어난 성능

1. 단어 벡터화

- 희소 표현(sparse representation) / 밀집 표현(dense representation)

- **희소 표현**: 원-핫인코딩, TF-IDF 등 빈도 기반 방법

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- 대부분의 벡터요소가 0으로 표현
- ex. 모델의 단어사전 5,000개 단어로 구성 → 10개의 토큰으로 이루어진 입력 텍스트 원- 핫인코딩으로 표현 → 최소 4,990개의 0이 포함된 벡터로 표현
- 단어 사전 크기 ↑ = 벡터 크기 ↑ ⇒ 공간적 낭비 발생
- 단어 간의 유사성 반영 X, 벡터 간 유사성 계산 비용 ↑
- **밀집 표현**: Word2Vec

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- 단어를 고정된 크기의 실수 벡터로 표현
⇒ 단어 사전 크기 ↑, 벡터 크기 고정
- 벡터 공간 상에서 단어 간의 거리를 효과적으로 계산
- 벡터의 대부분이 0이 아닌 실수 → 효율적 공간 활용
- 학습을 통해 단어를 벡터화 하기 때문에 단어의 의미 비교 가능

- 단어 임베딩 벡터(Word Embedding Vector) = 밀집 표현된 벡터

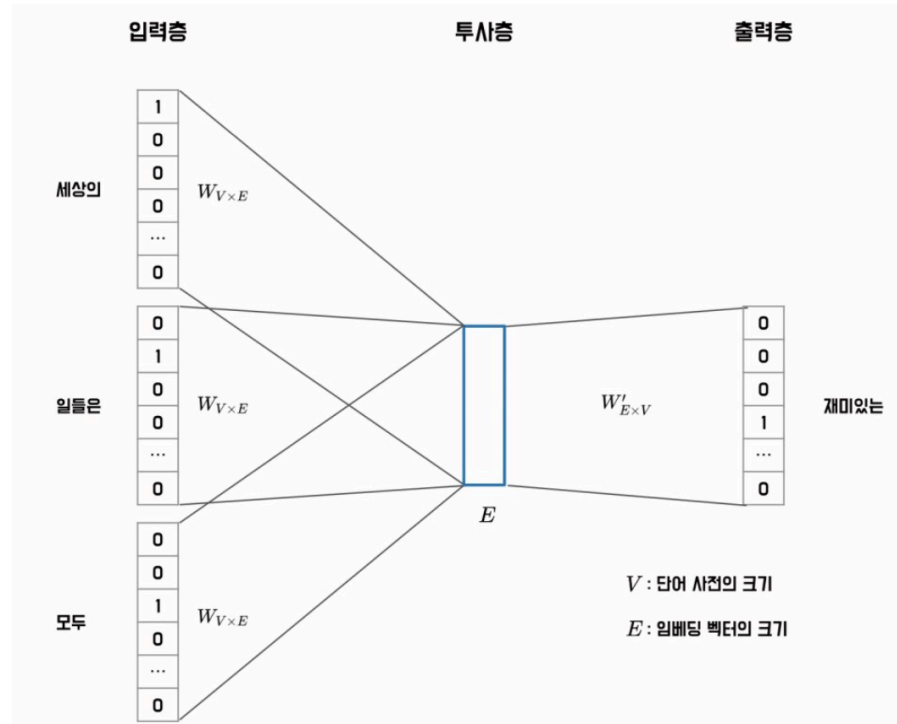
2. CBoW (Continuous Bag of Words)

: 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법

- 중심 단어: 예측해야 할 단어 / 주변 단어: 예측에 사용되는 단어
- 윈도우: 중심 단어를 맞추기 위해 고려할 주변 단어의 개수
- 슬라이딩 윈도우: 윈도우를 이동해가며 학습하는 방법
 - CBoW는 슬라이딩 윈도우를 사용해 한번의 학습으로 여러개의 중심 단어와 그에 대한 주변단어 학습
- ex. 하나의 입력 문장에서 윈도우 크기가 2일 때 학습데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

- 학습 데이터: (주변단어 | 중심단어)로 구성 → 대량의 말뭉치에서 효율적으로 단어의 분산표현 학습
- CBoW 모델 구조



- 각 입력 단어의 원-핫벡터를 입력값으로 받음
- 입력 문장 내 모든 단어의 임베딩 벡터를 평균내어 중심단어 임베딩 벡터 예측
- 입력단어는 원-핫벡터로 표현돼 투사층에 입력 및 통과
 - 투사층 : 원-핫벡터의 인덱스에 해당하는 임베딩 벡터 반환하는 순람표 (Lookuptable, LUT) 구조
- 각 단어는 E 크기의 임베딩 벡터로 변환
- 입력된 단어의 임베딩 벡터의 평균값 계산
- 계산된 평균 벡터를 가중치 행렬 $W(E \times V)$ 과 곱하면 V 크기의 벡터를 얻음
- 이 벡터에 소프트맥스 함수를 이용해 중심단어 예측

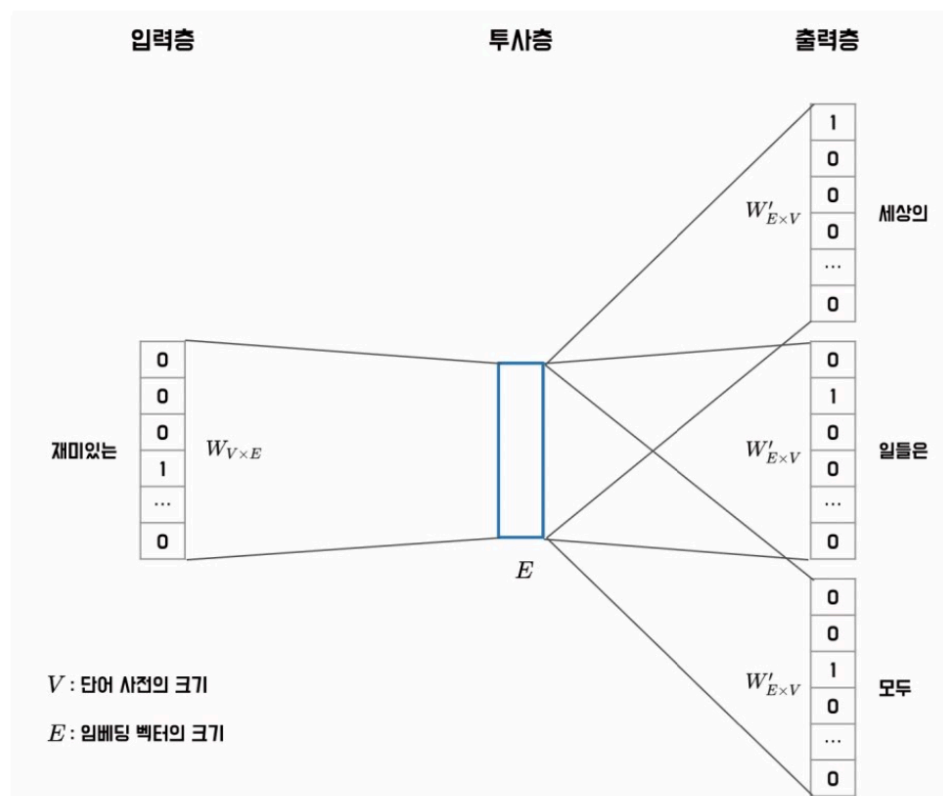
3. Skip-gram

: 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델 ($\leftrightarrow$ CBoW)

- 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터 세트 만들
 - 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델 학습
- ex. 윈도우 크기가 2일 때 학습데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

- 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터 만들어짐
(↔ CBoW : 하나의 윈도우에서 하나의 학습 데이터 만들어짐)
⇒ 더 많은 학습 데이터 추출 가능, CBoW보다 뛰어난 성능, 드물게 등장하는 단어를 더 잘 학습, 단어 벡터공간에서 더 유의미한 거리 관계 형성
- Skip-gram 모델 구조

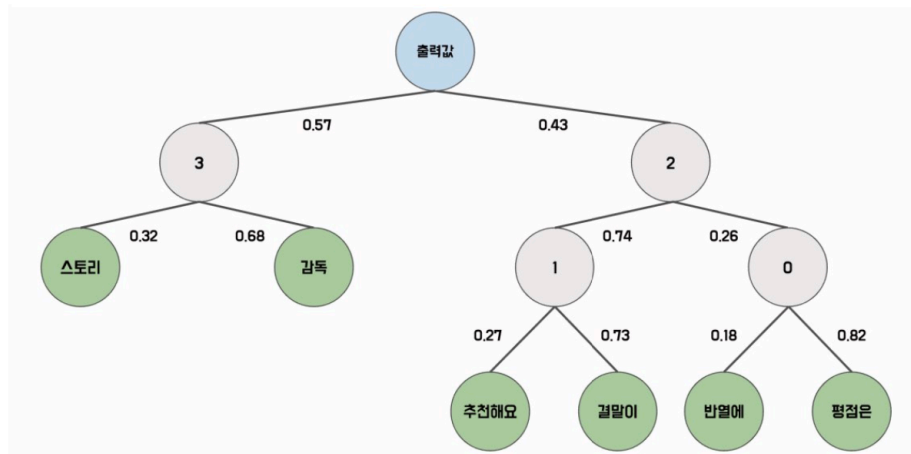


- 입력단어의 원- 핫벡터를 투사층에 입력해 해당 단어의 임베딩 데이터 가져옴 (= CBoW)
- 입력 단어의 임베딩과 가중치 행렬 $W(E \times V)$ 과의 곱셈 → V 크기의 벡터를 얻음
- 이 벡터에 소프트맥스 연산 취해 주변 단어 예측

- 소프트맥스 연산 : 모든단어를 대상으로 내적연산 수행
 - 말뭉치 크기 ↑ = 단일 사전 크기 ↑
 - ⇒ 대량의 말뭉치 통해 Word2Vec 모델 학습 시 학습 속도 ↓ : 단점
 - 이를 해결하기 위해 계층적소프트맥스 / 네거티브샘플링 기법 적용

4. 계층적 소프트맥스

- 출력층을 이진트리 구조로 표현해 연산 수행
- 자주 등장하는 단어 일수록 트리의 상위 노드에 위치 / 드물게 등장하는 단어 일수록 하위노드에 배치
- 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성
- 구조



- 각 노드는 학습이 가능한 벡터 가짐, 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수 통해 확률 계산
- 잎 노드 (Leaf Node): 가장 깊은 노드, 각 단어를 의미, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
- if 단어사전 크기 = V , 계층적 소프트맥스 시간복잡도 = $O(\log_2 V)$
 (↔ 일반적인 소프트맥스 연산 시간복잡도 = $O(V)$)

5. 네거티브 샘플링

: Word2Vec 모델에서 사용되는 확률적인 샘플링 기법, 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용

- 학습원도 내에 등장하지 않는 단어를 n개 추출하여 정답 단어와 함께 소프트맥스 연산 수행 → 전체 단어 확률 계산 필요 X, 효율적 학습 가능
 - 일반적으로 5~20개 단어 추출
 - 각 단어 추출 확률 수식

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

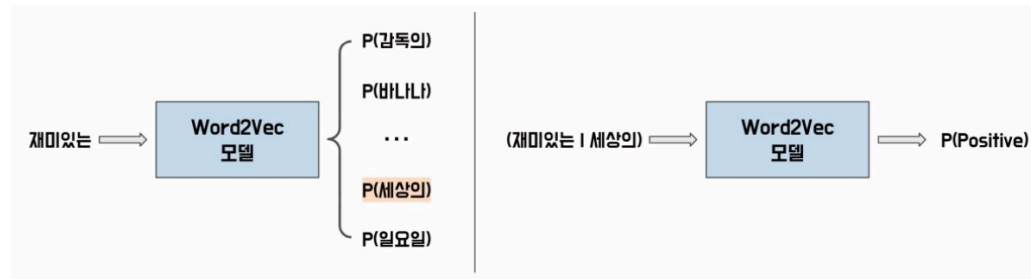
- $f(w_i)$ = 각 단어 w_i 의 출현 빈도 수
- $P(w_i)$ = 단어 w_i 가 네거티브 샘플로 추출될 확률
 - 출현 빈도 수에 0.75제곱한 값을 정규화 상수로 사용
- 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지 or 네거티브 샘플링으로 생성된 단어 쌍인지 이진분류
 - 로지스틱 회귀 모델 사용
- 모델의 학습과정에서는 추출할 단어의 확률분포를 구하기 위해 먼저 각 단어에 대한 가중치 학습
- 네거티브 샘플링 모델의 훈련데이터

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

→

입력 데이터	출력 데이터
재미있는. 세상의	1
재미있는. 일들은	1
재미있는. 모두	1
재미있는. 걸어가다	0
재미있는. 주십시오	0
재미있는. 미리	0
재미있는. 안녕하세요	0

- '재미있는' 이라는 중심단어를 통해 세상의 학습데이터 추출 → 실제 데이터에서 추출된 단어쌍은 1로, 네거티브 샘플링을 통해 추출된 가짜 단어 쌍은 0으로 레이블링
 - ⇒ 학습 목적: 다중분류 → 이진분류
- 소프트맥스 Word2Vec과 네거티브 샘플링 Word2Vec



- 입력단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블(1 또는 0)을 가져와 내적연산 수행
- 레이블이 1인 경우 해당 확률값 ↑, 레이블이 0인 경우 해당 확률값 ↓ : 모델의 가중치 최적화

6. 모델 실습: Skip-gram

- Word2vec모델 : 학습할 단어의 수 = V , 임베딩 차원 = E 로 설정 → W ($V \times E$) 행렬과 W' ($E \times V$) 행렬을 최적화 하며 학습
 - W ($V \times E$) 행렬 : 룩업 연산 수행 → 임베딩 클래스 사용 시 간편 구현 가능
 - 임베딩 클래스: 단어, 범주형 변수 등 이산 변수를 연속적인 벡터 형태로 변환해 사용, 연속적 벡터 표현은 모델이 학습하는 동안 단어의 의미와 관련된 정보 포착 → 단어 간의 유사도 계산
 - 룩업 : 이산 변수 값 → 연속적인 벡터 변환 과정
- 임베딩 클래스

```
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None,
    norm_type=2.0
)
```

- `num_embeddings` : 임베딩 수, 이산 변수의 개수 = 단어 사전의 크기
- `embedding_dim` : 임베딩 차원, 임베딩 벡터의 차원 수 = 임베딩 벡터의 크기
- `padding_idx` : 패딩 인덱스
 - 패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정
 - 병렬 처리를 위해 입력 배치의 문장 길이가 동일해야 하므로 입력문장들을 일정한 길이로 맞추는 역할

- 패딩 인덱스 < 임베딩 수
- 패딩인덱스의 벡터값은 모델학습 시 최적화되지 않음
- `norm_type` : 노름 타입, 임베딩 벡터의 크기를 제한하는 방법 선택
 - 기본값은 2 → L2 정규화 방식을 사용한다는 의미
 - if 노름 타입 = 1 → L1 정규화 방식 사용
- `max_norm` : 최대 노름, 임베딩 벡터의 최대 크기를 지정, 각 임베딩 벡터의 크기가 최대 노름값 이상이면 임베딩 벡터를 최대 노름 크기로 잘라내고 크기를 감소시킴

6.3 기본 skip-gram 클래스

- 기본 형식의 Skip-gram 모델 간단히 구현 가능 (↔ 계층적 소프트맥스 or 네거티브 샘플링과 같은 효율적인 기법 사용한 Skip-gram 모델)
- 입력 단어 & 주변 단어를 룩업테이블에서 가져옴 → 내적 계산 → 손실함수를 통해 예측 오차를 최소화 하는 방식으로 학습
- 기본형식의 Skip-gram모델 선언 → 모델학습에 사용할 데이터세트를 불러옴

6.4 영화 리뷰 데이터셋 전처리

- `corpus.test` : 테스트 세트 불러옴
- `okt` 토큰나이저 사용해 형태소 추출

6.5 단어 사전 구축

- Okt 토큰나이저 통해 토큰화된 데이터 활용
- `build_vocab` 함수 : 단어사전 구축
- `n_vocab` 매개변수 : 구축할 단어사전 크기
if 문서 내에 `n_vocab`보다 많은 종류의 토큰, 가장 많이 등장한 토큰 순서로 사전을 구축
- `special_tokens` : 특별한 의미를 갖는 토큰들
- `<unk>` 토큰 : OOV에 대응하기 위한 토큰, 단어 사전 내 없는 단어는 `<unk>` 토큰으로 대체

6.6 Skip-gram의 단어 쌍 추출

- `get_word_pairs` 함수: 토큰 입력 받아 Skip-gram 모델의 입력 데이터 사용할 수 있게 전처리
- `window_size` : 주변 단어를 몇개까지 고려할 것인지 설정, 각 문장에서는 중심 단어와 주변 단어를 고려하여 쌍 생성

- `idx` : 현재 단어의 인덱스
- `center_word` : 중심단어
- `window_start` / `window_end` : 현재 단어에서 얼마나 멀리 떨어진 주변 단어를 고려할 것인지를 결정 → 이때, `window_start` 와 `window_end`는 문장의 경계를 넘어가는 경우가 없게 조정
- 출력결과: 각 단어쌍은 [중심 단어, 주변 단어]로 구성, 임베딩 층은 단어의 인덱스를 입력으로 받음
 ∴ 단어 쌍 → 인덱스 쌍으로 변환 필요

6.7 인덱스 쌍 변환

- `get_index_pairs` 함수 : `get_word_pairs` 함수에서 생성된 단어쌍 → 토큰 인덱스 쌍 변환
- `token_to_id` : 앞서 생성한 `word_pairs`와 단어에 해당하는 ID를 매핑한 딕셔너리 → 인덱스쌍 생성
 → `get` 메서드 : 토큰이 단어사전 내에 있으면 해당 토큰 인덱스 반환 / 없다면 <unk> 토큰 인덱스 반환
 ⇒ 생성된 인덱스 쌍은 Skip-gram모델의 입력 데이터로 사용

6.8 데이터로더 적용

- `index_pairs` : `get_index_pairs` 함수에서 생성된 중심단어와 주변단어 토큰의 인덱스 쌍으로 이루어진 리스트
 → 텐서 형식으로 변환 : 텐서는 [N, 2]의 구조, 중심단어와 주변단어로 나눠 데이터 세트로 변환

6.9 Skip-gram 모델 준비 작업

- `Vanillaskipgram` 클래스
 - `vocab_size` : 단어 사전 크기 → 전체 단어 집합의 크기 전달
 - `embedding_dim` : 임베딩 크기, 128 할당
- 손실함수 : 단어사전 크기만큼 클래스가 있는 분류문제 → 교차 엔트로피 사용
 교차 엔트로피 : 내부적으로 소프트맥스 연산 수행 → 신경망의 출력값 후처리 없이 활용 가능

6.10 모델 학습

- 모델 학습이 완료 되면 $W(V \cdot E)$ 행렬과 $W'(E \cdot V)$ 행렬 중 하나의 행렬 선택 → 임베딩값 추출

6.11 임베딩 값 추출

- Word2Vec 모델의 임베딩 행렬 이용 → 각 단어의 임베딩 값 매핑 → 인덱스 30 값의 단어와 임베딩값 출력
⇒ 단어 간의 유사도 확인 가능
- 임베딩의 유사도 측정 방법: 코사인 유사도 (가장 일반적)
 - 코사인 유사도: 두 벡터 간의 각도 이용하여 유사도 계산, 두 벡터가 유사할수록 값이 1에 가까워지고, 다를수록 0에 가까워짐

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- = (두 벡터 / 내적인 벡터의 크기의 곱)
 - a,b : 유사도를 계산하려는 벡터
 - 벡터의 크기: 각 성분의 제곱합에 루트 씌운 값
- 임베딩 공간에서 단어 간의 유사도를 측정하는 데 매우 유용

6.12 단어 임베딩 유사도 계산

- `cosine_similarity` 함수 : 입력 단어와 단어 사전 내의 모든 단어와의 코사인 유사도 계산
 - a 매개변수 : 임베딩 토큰
 - b 매개변수: 임베딩 행렬
→ 임베딩 행렬은 [5001, 128]의 구조 → 노름 계산 시 axis=1 방향으로 계산
- `top_n_index` 함수 : 유사도 행렬을 내림차순으로 정렬해 어떤 단어가 가장 가까운 단어인지 반환
 - 입력 단어도 단어 사전에 포함 → 입력 단어 자신이 가장 가까운 단어가 됨.
∴ 두 번째 가까운 단어 부터 계산해 반환

7. 모델 실습: Gensim

- 매우 간단한 구조의 Word2Vec 모델 : 학습할 때 데이터 수가 적은 경우에도 학습하는 데 오랜 시간이 소요
→ 계층적 소프트맥스나 네거티브 샘플링 같은 기법 사용 → 더 효율적으로 학습 가능
→ 젠심 (Gensim) 라이브러리 사용 시 Word2Vec과 같은 자연어 처리 모델 쉽게 구성
- 젠심 라이브러리

- 대용량 텍스트 데이터의 처리를 위한 메모리, 효율적인 방법을 제공 → 대규모 데이터 세트에서도 효과적으로 모델 학습 가능
- 학습된 모델 저장하여 관리 가능
- 비슷한 단어 찾기 등 유사도와 관련된 기능 제공
- 젠심 라이브러리 Word2Vec 클래스

```
word2vec = gensim.models.Word2Vec(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window=5,
    min_count=5,
    workers=3,
    sg=0,
    hs=0,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000
)
```

- **sentences** : 입력 문장, 모델의 학습 데이터, 토큰 리스트로 표현 → 학습 문장들의 리스트로 이루어짐
- **corpus_file** : 말뭉치 파일, 학습 데이터를 파일로 입력 할 때 파일의 경로, 입력 문장 대신에 사용
- **vector_size** : 학습할 임베딩 벡터의 크기, 임베딩 차원 수 설정
- **alpha** : Word2Vec 모델 학습률
- **window** : 학습 데이터 생성할 윈도우의 크기
 - if window = 3 , 중심단어로부터 3만큼 떨어진 단어까지 주변단어로 고려해 데이터 생성
- **min_count** : 학습에 사용할 단어의 최소빈도, 말뭉치 내에 최소 빈도만큼 등장하지 않은 단어는 학습에 사용 X
- **max_final_vocab** : 최대 최종 단어사전, 단어 사전의 최대크기,

- if 최소 빈도를 충족하는 단어가 최대 최종 단어사전 보다 많으면 자주 등장한 단어 순으로 단어사전 구축
- `workers` : 작업자 수, 빠른 학습을 위해 병렬 학습할 스레드 개수
- `sg` : Skip-gram 모델 사용 여부 설정, 1 = Skip-gram, 0 = CBoW 사용
- `hs` : 계층적 소프트맥스 사용 여부 설정, 1 = 사용 0 / 0 = 사용 X
- `chow_mean` : CBoW 모델 구성 시 사용되는 하이퍼파라미터로 중심 단어와 주변 단어 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부 설정, 1 = 평균화 0 / 0 = 평균화 X
- `negative` : 네거티브 샘플링의 단어 수, 0 = 네거티브 샘플링 사용 X
- `ns_exponent` : 네거티브 샘플링 확률의 지수
- `epochs` : 학습 에폭 수
- `batch_words` : 몇 개의 단어로 학습 배치 구성할지 결정

6.13 Word2Vec 모델 학습

- VaniLlSkipgram 클래스에 비해 매우 빠르게 학습
- word2vec 인스턴스는 파이토치의 저장 메서드와 동일한 방법으로 저장 가능
- *. model 과 같은 형식으로 저장 ↔ 파이토치 모델 파일

6.14 임베딩 추출 및 유사도 계산

- word2vec 인스턴스의 w속성 : 학습된 단어 벡터 모델을 포함한 WordVecKeyedVectors 객체 반환
- 단어 벡터 검색과 유사도 계산 등의 작업을 수행하는 데 사용
- 주어진 단어에 대해 가장 유사한 단어를 찾는 `most_Similar` 메서드, 두 단어 간의 유사도를 계산하는 `Similarity` 메서드 제공

⇒ Word2Vec : 분포가설을 통해 쉽고 빠르게 단어의 임베딩을 학습 가능 but, 단어의 형태학적 특징을 반영하지 못한다는 한계 있음

3 fastText

- 2015년 메타의 FAIR연구소에서 개발한 오픈소스 임베딩 모델
- 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘
- 단어와 문장을 벡터로 변환하는기술을 기반으로 하며, 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용

- 벡터화 기술은 Word2Vec과 유사하지만, fastText는 단어의 하위단어를 고려 → 더 높은 정확도와 성능 제공
 - 하위단어를 고려하기 위해 N-gram 사용 → 단어를 분해하고 벡터화하는 방법으로 동작
- 단어의 벡터화를 위해 <, > 와 같은 특수 기호를 사용하여 단어의 시작과 끝을 나타냄, 기호는 단어의 하위 문자열 고려하는 데 중요한 역할
- 기호가 추가된 단어는 N-gram을 사용하여 **하위 단어 집합**으로 분해됨
 - 분해된 하위 단어집합에는 나뉘지지 않은 단어 자신도 포함, 단어 집합이 만들어지면 각 하위단어는 고유한 벡터값을 갖게 됨

입력 단어	< > 더하기	N-gram 분해	전체 단어 추가
서울특별시	<서울특별시>	<서울 서울특 울특별 특별시 별시>	<서울 서울특 울특별 특별시 별시> <서울특별시>

- fastText 입력 토큰 처리 과정
 1. 토큰의 양 끝에 <, > 를 붙여 토큰의 시작과 끝을 인식할 수 있게 한다.
 2. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해한다. 트라이 그래프를 사용한다면, '서울특별시'는 [서울, 서울특, 울특별, 특별시, 별시] 로 분해된다.
 3. 분해된 하위단어 집합에는 나뉘지지 않은 토큰 자체도 포함한다. 이렇게 하위 단어 집합이 만들어지면, 각 하위단어는 고유한 벡터값을 갖는다.
 - 각 하위단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터 계산
- fastText는 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 단어 벡터를 구성 → 단어의 부분 문자열을 고려하는 유연하고 정확한 하위 단어 집합 생성
 - = 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습 가능 → 비슷한 단어 끼리 비슷한 임베딩 벡터를 갖게 되어, 단어 간 유사도를 높일 수 있음
 - OOV 단어도 하위 단어로 나누어 임베딩 계산 가능
 - = 말뭉치에 등장하지 않은 단어라도 유사한 하위단어를 가지고 있으면 유사한 임베딩 벡터 갖게 됨

1. 모델 실습

- fastText 모델도 CBoW와 Skip-gram으로 구성, 네거티브 샘플링 기법 사용해 학습
- fastText 모델은 하위 단어로 학습 ↔ Word2Vec 모델은 기본 단위로 학습
 - Word2Vec 보다 복잡한 과정 필요, 모든 하위 단어 크기를 갖는 임베딩 계층이 필요
 - 젠심 라이브러리는 이러한 복잡한과정을 FastText 클래스로 제공
- fastText 클래스

```
fasttext = gensim.models.FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

- Word2Vec과 기술적인 공통점이 많으므로 Word2Vec의 매개변수 대부분 공유
 - N-gram 범위를 결정하는하이퍼파라미터 추가
 - `min_n` : N-gram의 최솟값
 - `max_n` : N-gram의 최댓값

6.15 KorNLI 데이터세트 전처리

- KorNLI 데이터세트: 한국어 자연어 추론을 위한 데이터 세트
- 자연어 추론 : 두 개이상의 문장이 주어졌을 때, 두 문장 간의 관계를 분류하는 작업
- 주어진 문장이 함의 관계(entailment), 중립 관계(neutral), 불일치 관계(contradiction) 중 어느 관계에 해당되는지 분류 가능
- KorNLI 인스턴스는 `get_all_texts` 와 `get_all_pairs` 메서드 제공

- `get_all_texts` : 모든 문장 튜플 형태로 반환
- `get_all_pairs` : 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환
- fastText 모델은 입력 단어의 구조적 특징 학습 가능 ($\leftrightarrow$ Word2Vec)
 - 형태소 분석기를 통해 토큰화 X, 띄어쓰기를 기준으로 단어를 토큰화 해 학습 진행

6.16 fastText 모델 실습

6.17 fastText OOV 처리

- `wv.index_to_key` 메서드 : 학습된 단어사전을 나타내는 리스트
- WordVec 모델 : 단어 사전에 존재하지 않는 단어 임베딩 계산 불가
 - $\leftrightarrow$ fastText는 하위 단어로 나뉘어 있기 때문에 '사랑해요' 등의 단어 처리 가능
 - '사랑해요' 토큰은 '사랑', '랑해', '해요' 등의 하위단어로 분해 됨
 - 이 분해된 하위 단어의 임베딩을 모두 합해 전체 토큰의 임베딩 계산
 - 다른 단어에서 등장했던 '사랑'이나 '해요'와 같은 하위 단어 통해 '사랑해요' 토큰의 임베딩 계산

⇒ fastText는 OOV문제를 효과적으로 해결 가능. (특히, 한국어)

4 순환 신경망

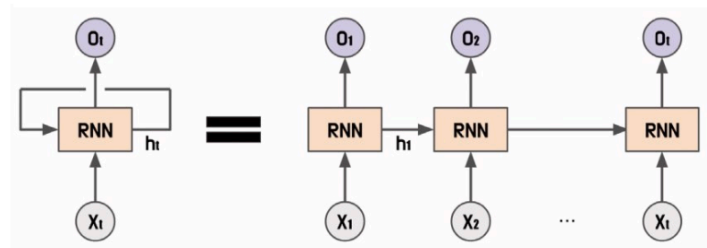
- 순서가 있는 연속적인 데이터를 처리하는 데 적합한 구조
- 각 시점(Time step)의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동
- 연속형 데이터: 특정 시점 t에서의 데이터가 이전 시점(t0, t1, ... tn-1)의 영향을 받는 데이터
- 자연어 데이터 : 문장안에서 단어들이 순서대로 등장하므로, 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장 의미를 형성 → 연속적인 데이터의 일종
 - t 번째 단어는 t-1번째 까지의 단어에 영향을 받아 결정됨
 - 긴 문장일수록 앞 선 단어들과 뒤따르는 단어들 사이에 강한 상관관계 존재

→ 자연어 처리에서는 이러한 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요

1. 순환 신경망

- 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류
- 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용

- 시계열데이터, 자연어처리. 음성인식 및 기타 시퀀스 데이터와 같은 도메인에서 널리 사용
 → 이러한 데이터는 일반적으로 길이가 가변적이며, 순서에 따라 의미가 있기 때문에, 순환신경망으로 처리하기에 적합한 구조
- 연속형 데이터를 순서대로 입력 받아 처리하며 각 시점마다 은닉상태의 형태로 저장
- 셀(Cell) : 각 시점의 데이터를 입력으로 받아 은닉상태와 출력값을 계산하는 노드
 → 이전 시점의 은닉상태 $h(t-1)$ 을 입력으로 받아 현재 시점의 은닉상태 $h(t)$ 계산
 → 이러한 재귀적 특징으로 인해 '순환'신경망이라 불림



- 순환 신경망의 은닉상태 수식

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- $\sigma(h)$: 순환 신경망의 은닉상태를 계산하기 위한 활성화 함수
 → 이전 시점 t-1의 은닉상태 $h(t-1)$ 과 입력값 x_1 를 입력 받아 현재 시점의 은닉상태 $h(t)$ 를 계산
 - 가중치(W)와 편향(b)을 이용해 계산
 - $W(hh)$: 이전 시점t-1의 은닉상태 $h(t-1)$ 에 대한 가중치
 - $W(xh)$: 입력값 x_1 에 대한 가중치
 - $b(h)$: 은닉상태 $h(t)$ 의 편향
- 순환 신경망의 출력값

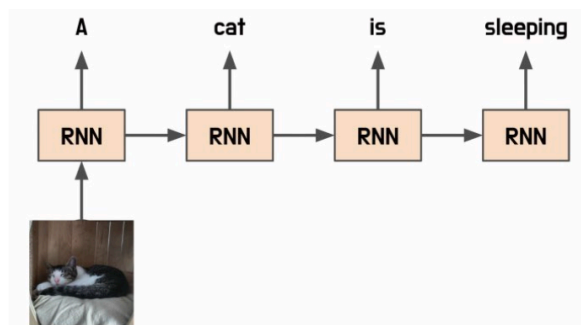
$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- $\sigma(y)$: 순환 신경망의 출력값을 계산하기 위한 활성화 함수
 - 현재시점의 은닉상태 $h(t)$ 를 입력으로 받아 출력값 y_1 을 계산
 - 가중치(W)와 편향(b)을 이용해 계산
 - $W(h_t)$: 현재시점의 은닉상태 $h(t)$ 에 대한 가중치
 - $b(y)$: 출력값 y_1 의 편향
- 순환 신경망 구조
 - 일대다 구조

: 하나의 입력 시퀀스에 대해 여러 개의 출력 값을 생성하는 순환 신경망 구조

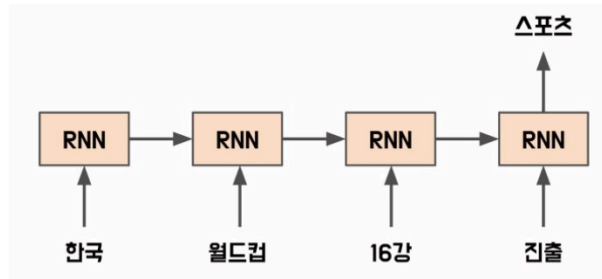
 - 자연어 처리 분야에서는 일대다구조를 사용하여 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측하는 작업 가능
 - 입력 시퀀스 - 문장으로 구성, 출력시퀀스 - 각 단어의 품사로 구성
 - 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 이미지 캡셔닝 모델이 됨
 - 입력 시퀀스 - 이미지로 구성, 출력시퀀스 - 이미지에 대한 설명 문장
 - 일대다구조를 구현하기 위해서는 출력시퀀스의 길이를 미리 알고 있어야 함
 - 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이를 예측하는 모델을 함께 구현해야 함



- 다대일 구조

: 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조

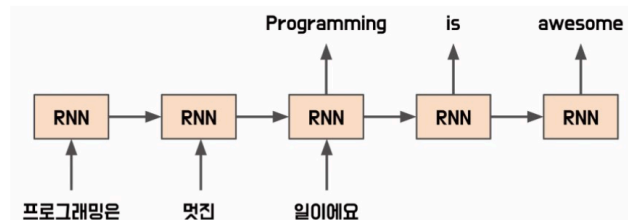
 - 감성분류 분야에서는 다대일 구조를 사용하여 특정문장의 감정 예측 작업 가능
 - 입력 시퀀스 - 문장, 출력시퀀스 - 해당 문장의 감정(긍정, 부정)
 - 입력 시퀀스가 어떤 범주에 속하는지 구분하는 문장분류, 두 문장 간의 관계를 추론하는 자연어 추론 등에 적용 가능



◦ 다대다 구조

: 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조

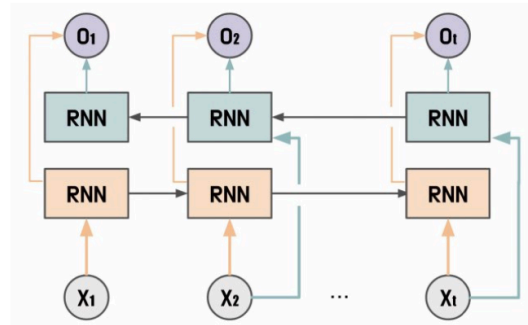
- 다양한 분야에서 활용, 입력문장에 대해 번역된 출력 문장을 생성하는 번역기, 음성 인식 시스템에서 음성 신호를 입력으로 받아 문장을 출력하는 음성인식기 등에 서 사용
- 입력시퀀스와 출력시퀀스의 길이가 서로 다른 경우가 있음
 - ex. 입력 문장의 길이와 출력 문장의 길이가 일치하지 않는 경우
 - 입력시퀀스와 출력시퀀스의 길이를 맞추기 위해 패딩을 추가하거나 잘라내는 등의 전처리 과정이 수행됨
- 시퀀스- 시퀀스(Seq2Seq) 구조 : 입력시퀀스를 처리하는 인코더와 출력시퀀스를 생성하는 디코더로 구성



◦ 양방향 순환 신경망

: 기본적인 순환신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식

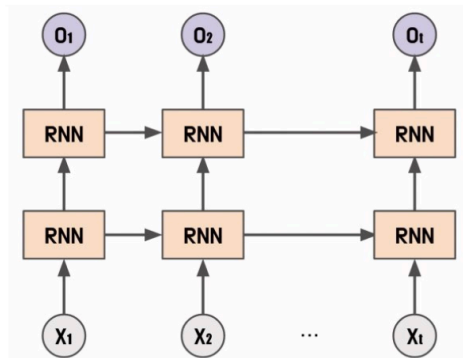
- 순환 신경망 : 현재 시점의 입력값을 처리하기 위해 이전 시점 $t-1$ 의 은닉상태 $h(t-1)$ 이용
 - ↔ 양방향 순환 신경망 : 이전 시점 $t-1$ 의 은닉상태 뿐만 아니라 이후 시점 $t+1$ 의 은닉 상태로 함께 이용
- t 시점 이후의 데이터도 t 시점의 데이터를 예측하는 데 사용될 수 있음
 - 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로 거꾸로 읽어 들여 처리하는 방식으로 이루어짐



○ 다중 순환 신경망

: 여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환신경망이 서로 다른 정보를 처리하도록 설계 돼 있음

- 여러개의 순환신경망층으로 구성되며, 각 층에서의 출력값은 다음 층으로 전달되어 처리
- 여러 개의 층으로 구성된 RNN은 데이터의 다양한 특징을 추출할 수 있어 성능 향상 + 층이 깊어질수록 더 복잡한 패턴을 학습 할 수 있음
- but 순환신경망 층 ↑ → 학습시간 ↑, 기울기 소실 문제 발생 가능성 ↑

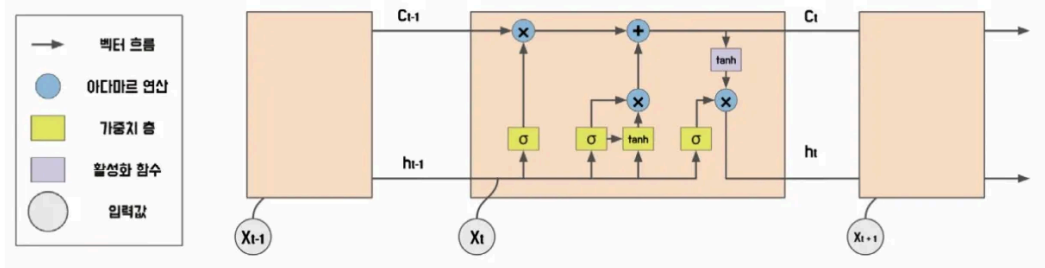


2. 장단기 메모리

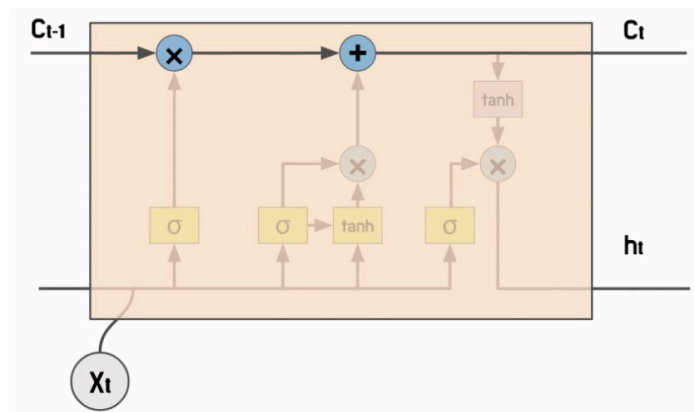
- 기존 순환신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 모델
- 순환신경망의 경우 시간적으로 연속된 데이터를 다룰 수 있다는 장점이 있지만, 앞선 시점에서의 정보를 끊임없이 반영하기에 학습데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달되지 않는다는 단점 → 장기 의존성 문제 발생
- 활성화 함수로 사용되는 하이퍼볼릭탄젠트 함수나 ReLU 함수의 특성으로 인해 역전과 과정에서 기울기 소실이나 기울기 폭주가 발생할 가능성

⇒ 장단기메모리는 순환신경망과 비슷한 구조를 가지지만, 메모리 셀(Memorycell)과 게이트(Gate)라는 구조를 도입해 장기 의존성 문제와 기울기 소실 문제 해결

- 장단기 메모리 구성



- 셀 상태(Cell state)와 망각 게이트(Forget gate), 기억 게이트(Input gate), 출력 게이트(Output gate)로 정보의 흐름을 제어
- 셀 상태 : 정보를 저장하고 유지하는 메모리 역할, 출력 게이트와 망각 게이트에 의해 제어됨
- 망각 게이트: 장단기 메모리에서 이전 셀 상태에서 어떠한 정보를 삭제할지 결정하는 역할, 현재 입력과 이전 셀상태를 입력으로 받아 어떤 정보를 삭제할지 결정
- 입력 게이트 : 새로운 정보를 어떤 부분에 추가할지 결정하는 역할, 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정
- 출력 게이트 : 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할을 하며 현재 입력과 이전 셀 상태, 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정
- 장단기 메모리 구조



- 메모리 셀 : 순환 신경망의 은닉 상태와 유사하게 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할
- 순환 신경망에서 은닉 상태는 출력값을 계산하는 데 사용되지만, 장단기 메모리의 메모리 셀은 출력값 계산에 직접 사용되지 않음
 - 대신 장단기 메모리는 망각게이트, 입력게이트, 출력게이트를 통해 어떤 정보를 버릴지, 어떤 정보를 기억할지, 어떤 정보를 출력할지를 결정하는 추가 연산 수행

→ 세가지 게이트는 모두 활성화 함수로 시그 모이드 사용

- 시그모이드 함수는 입력 값을 0과 1 사이의 값으로 변환하므로 게이트의 출력 값은 각각 0에서 1사이의 값으로 결정

→ 이 값이 해당게이트가 입력값에 대해 얼마나 많은 정보를 통과시킬지 결정

ex. if 망각게이트의 출력값 = 1, 이전 시점의 기억 상태가 현재 시점의 기억 상태에 완전히 유지, 0이면 이전 시점의 기억 상태는 현재 시점의 기억 상태에 전혀 반영 되지 않음

1. 망각 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 메모리 셀을 곱한 값으로 이전 시점의 메모리 셀 업데이트

→ 시그모이드 함수를 거치면 0과 1사이의 값이 출력되며, 이 값은 어떤 정보를 유지할 것인지, 아니면 망각할 것인지를 결정

2. 기억 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 하이퍼볼릭 탄젠트 함수를 거친 값의 곱으로 새로운 기억 값을 계산

→ 시그모이드 함수는 입력 값의 중요도를 결정,하이퍼볼릭 탄젠트 함수는 입력값을 - 1과 1 사이의 값으로 변환

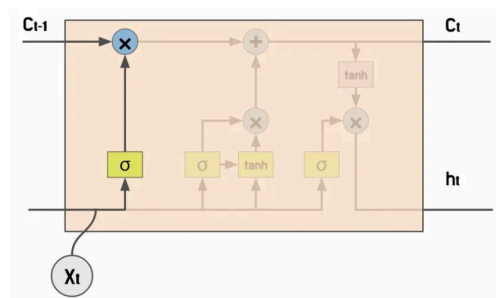
∴ 기억 게이트는 어떤정보를 얼마나 추가할지를 결정

3. 출력 게이트 : 현재 시점의 입력과 이전 시점의 은닉 상태, 그리고 새로 업데이트 된 메모리셀을 입력으로 받아 현재 시점의 출력값을 계산

∴ 출력 게이트는 어떤 정보를 출력할지를 결정

⇒ 게이트들은 입력 값에 대한 가중치를 동적으로 조절, 적절한 정보만을 기억하고 유지할수 있다는 장점 가짐

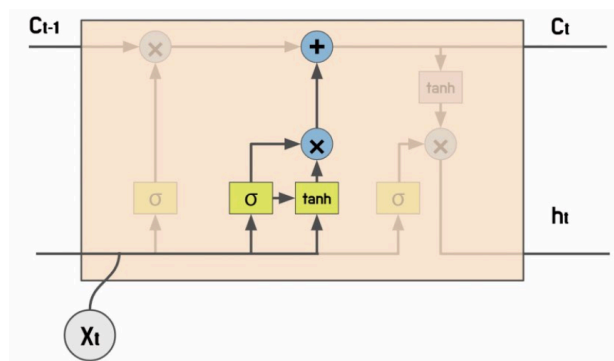
○ 망각게이트



- 이전 시점의 은닉상태 $h(t-1)$ 과 현재 시점의 입력값 x_1 를 통해 연산 수행
- 수식

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

- 이전 시점의 메모리셀과 현재 시점의 입력 바탕으로 어떤 정보를 삭제 할지 결정
- 출력값 $f(t)$ 는 시그모이드 활성화 함수를 사용해 계산
- $W(f)x$, $W(f)h$: 입력값과 은닉상태를 위한 가중치
 - 두 가중치를 통해 망각게이트 출력값을 최적화
- $b(f)$: 망각게이트의 편향
- 망각게이트 출력값은 메모리셀을 계산하기 위한 가중치로 사용
- 출력값은 0에서 1사이의 값
 - 이 값이 1에 가까우면 더 많은 정보를 유지, 0에 가까우면 더 많은 정보 삭제, 정확히 1이라면 아무런 정보를 삭제하지 않음, 0이 라면 모든 정보를 삭제
- 기억 게이트



■ 수식

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

- $g(i)$ 와 $i(t)$ 로 구성돼 있으며, 망각 게이트와 동일한 연산으로 계산
- $g(i)$: 활성화 함수로 하이퍼볼릭 탄젠트 사용 / $i(t)$: 시그모이드 사용
- $g(i)$: - 1 에서 1 사이의 값을 가지므로 이전 시점의 은닉 상태와 현재 시점의 입력값은 모두 $[- 1, 1]$ 범위 안에 존재

→ $g(i)$ 만으로는 새로운 은닉상태를 계산하기 위해 이 은닉상태를 얼마나 기억할지 제어하기 어려움

- $i(t)$: $[0,1]$ 의 값을 가지므로 현재시점에서 얼마나 많은 정보를 기억할 것인지를 결정하는 가중치 역할, 1에 가까울수록 기억할 정보가 많아지고, 0에 가까울수록 정보 망각

→ $i(t)$ 값으로 새로운 은닉 상태의 기억 제어

○ 메모리 셀

- 망각 게이트와 기억 게이트의 정보로 현재 시점의 메모리셀 값을 계산
- 수식

$$c_t = f_t \odot c_{t-1} + g_t \odot i_t$$

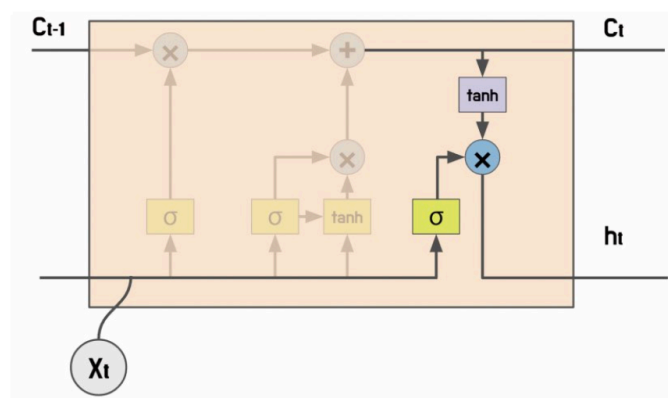
- $f(t)$: 망각 게이트 출력값
- $C(t-1)$: 이전 시점의 메모리 셀 값

→ 이 값을 원소별 곱셈연산 = 아다마르 곱 → 현재 시점의 메모리 셀 값

⇒ 망각 게이트 값 $f(t)$ 가 0에 가까울수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향을 미치지 않게 됨

- 기억 게이트도 아다마르 곱을 통해 계산되며 망각게이트와 합산됨
- 망각 게이트는 이전 시점의 메모리 셀을 얼마나 유지할지 결정, 기억게이트는 현재 시점의 새로운 정보를 얼마나 받아들일지 결정

○ 출력 게이트



- 수식

$$o_t = \sigma(W_x^{(o)} x_t + W_h^{(o)} h_{t-1} + b^{(o)})$$

- 현재 시점의 은닉상태 제어, 출력 게이트도 망각게이트에서 사용된 수식과 동일한 수식 사용, 활성화 함수로 시그모이드 사용
- 시그모이드 함수는 입력값을 [0, 1] 범위로 변환하므로 현재 시점의 은닉 상태 값을 얼마나 사용할지 결정
- 출력 게이트와 현재 메모리셀을 연산한다면 새로운 은닉 상태 계산 가능
- 현재 은닉 상태 계산

$$h_t = o_t \odot \tanh(c_t)$$

- 현재 시점의 은닉상태 h(t)는 출력 게이트와 하이퍼볼릭 탄젠트를 적용한 메모리 셀 값으로 계산
- 출력 게이트는 [0, 1]의 범위를 가지며, 하이퍼볼릭 탄젠트가 적용된 메모리셀은 [- 1, 1] 범위를 가짐

⇒ 두 값을 아다마르 곱 연산하면 현재 시점의 은닉 상태가 이전 시점의 은닉 상태에서 얼마나 영향을 받는지 계산 가능

→ 장단기 메모리는 현재 은닉상태에서 이전 은닉 상태의 정보의 중요도 제어 가능

- 장단기 메모리 클래스

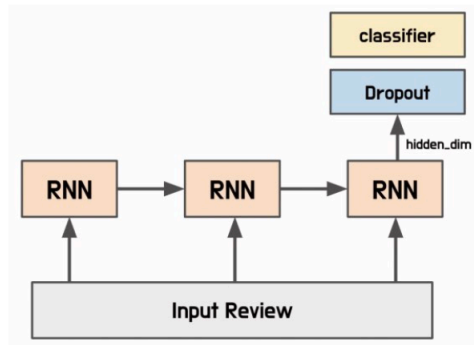
```
lstm = torch.nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size=0
)
```

- 순환 신경망 클래스와 거의 유사한 구조

- 활성화 함수를 명확하게 정의해 사용하므로 순환 신경망에서 사용하는 활성화 함수 매개 변수 사용X
- `proj_size` : 투사 크기 매개변수, 장단기 메모리 계층의 출력에 대한 선형 투사 크기 결정
 - 투사크기 > 0 : 은닉 상태를 선형투사를 통해 다른 차원으로 매핑
 - 투사크기 = 0 : 은닉 상태 차원 변환 X 그대로 유지
- 출력 차원 조절 시, 투사 크기 매개변수로 모델의 크기와 복잡도 변경, 투사크기는 은닉 상태 크기보다 작은 값으로 설정
- `inputs` : 순전파 연산 시 입력값 → `h_0` : 초기 은닉 상태 → `c_0` : 초기 메모리 셀 상태 ⇒ 순전파 연산 수행
 - ⇒ `outputs` : 출력값, `h_n` : 최종 은닉 상태, `c_n` : 초기 메모리 셀 상태 반환
- 입력 값 차원 : 앞선 순환 신경망 클래스에서 사용하던 입력값 차원 형태와 동일한 형태로 [배치크기, 시퀀스길이, 입력특성크기] 전달
- 초기 은닉 상태: 순환 신경망 클래스와 동일한 구조를 갖지만, 장단기 메모리는 선형 투사를 통해 출력층의 차원을 변경할 수 있으므로 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력
- 초기 메모리 셀 : 순환 신경망 클래스의 초기 은닉 상태와 동일한 구조인 [계층수 x 양방향 여부+1, 배치 크기, 은닉 상태 크기] 를 가짐, 선형투사를 적용하더라도 은닉 상태 크기 사용
- 순방향 연산 결과 : 출력값과 최종 은닉 상태, 최종 메모리 셀 상태가 반환됨, 출력값은 수식도 순환 신경망과 동일한 수식을 사용하지만, 선형 투사가 적용됐으므로[배치크기, 시퀀스길이,(양방향여부 +1) x 투사 크기(또는 은닉 상태 크기)]로 구성
- 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개 변수 설정에 주의, 메모리셀의 상태도 입력으로 전달해야하며, 은닉 상태와 메모리셀 상태를 튜플로 묶어 사용하므로 사용에 주의

3. 모델 실습

- 금/부정 분류 모델 구조



- 임베딩: 임베딩 벡터를 가져오는 임베딩 계층, [어휘사전의 크기X임베딩 벡터의 크기] 순람표 구조
- 임베딩 계층: 입력 테스트를 정수 인코딩 한 뒤, 해당 색인 값에 해당하는 임베딩을 가져오는 역할, 초기값으로는 무작위 값 할당 후 학습 통해 임베딩 계층이 최적화 되게 만들거나, 사전 학습된 임베딩 벡터를 가져와 사용

6.20 문장 분류 모델

- `SentenceClassifier` 클래스
 - 임베딩 층을 구성할 때 사용하는 단어 사전 크기 `n_vocab` 와 순환신경망 클래스와 장단기 메모리 클래스에서 사용하는 매개변수를 입력으로 전달받음
 - 모델 종류 `model_type` 매개변수로 순환신경망을 사용할지, 장단기 메모리를 사용할지 설정
- 초기화 메서드 : `SentenceClassifier` 클래스에 입력된 함수의 매개변수에 따라 모델 구조 미세 조정
- 순방향 메서드: 입력받은 정수 인코딩을 임베딩 계층에 통과 → 임베딩 값 얻음 → 모델에 입력하여 출력값 얻음 → 출력값(`output`)의 마지막 시점만 활용할 예정, `[ :, -1, : ]`으로 마지막 시점의 결괏값만 분리 → 분류기 계층에 전달

6.21 데이터셋 불러오기

- `corpus.test` 로 테스트 세트를 불러오고, 이 데이터 세트를 학습 데이터와 테스트 데이터로 분리

6.22 데이터 토큰화 및 단어 사전 구축

- Okt 토큰라이저를 사용해 단어를 토큰화하고 단어사전 구축
- 문장의 길이를 맞추기 위해 `<pad>` 토큰을 `special_tokens` 에 추가

6.23 정수 인코딩 및 패딩

- 학습 데이터 세트와 테스트 데이터 세트를 단어 사전에 있는 정수로 인코딩
→ 정수로 인코딩하면 모델이 텍스트 데이터를 처리하기 쉬워짐

- 최대 길이 `max_length` 는 입력 테스트의 길이를 고려해 결정
 - 너무 클 경우 : 입력 행렬 크기 ↑ 리소스 ↑
 - 너무 작을 경우: 문장 반영 ↓ 모델 성능 ↓
- `pad_sequences` 함수 : 너무 긴 문장은 최대길이로 줄이고, 너무 작은 길이라면 최대 길이와 동일한 크기로 변환 → 시퀀스를 최대 길이로 잘라내고, 시퀀스 길이가 작으면 `<pad>` 토큰을 시퀀스 뒤에 붙여 동일한 길이로 변경
- OOV의 경우 `1(<unk>)` 로 인코딩, 시퀀스 길이가 짧은 경우 최대 길이(32) 가 될 수 있게 `0(<pad >)` 토큰이 추가됨

6.24 데이터로더 적용

- `TensorDataset` 클래스 : 파이토치 텐서 형태를 입력값으로 받음 → 정수 인코딩과 라벨값을 파이토치 텐서 형태로 변환 → 변환된 데이터세트는 데이터로더 클래스에 적용

6.25 손실 함수와 최적화 함수 정의

- `BCEwithLogitsLoss` 클래스 : `BCELoss` 클래스 + `Sigmoid` 클래스, `BCELoss`클래스를 사용하는 경우, `criterion(torch.sigmoid(output), target)` 과 동일한 형태로 간주
- 최적화 함수는 `RMSProp(Root Mean Square Propagation)` 을 적용
 - `RMSProp` : 모든 기울기를 누적하지 않고, 지수 가중 이동 평균 사용해 학습률 조절

6.26 모델 학습 및 테스트

- 에폭마다 테스트 데이터세트로 모델의 검증 손실과 검증 정확도 확인
- 모델 학습 과정에서 임베딩 계층을 비롯한 순환신경망 내의 여러 가중치가 최적화 됨
- 학습된 임베딩 계층의 가중치를 동일한 단어사전을 사용하는 토큰의 임베딩 값으로 사용할수도 있음

6.27 학습된 모델로부터 임베딩 추출

- 긍정/부정 분류는 임베딩 계층이 아닌 순환신경망의 연산이 더 중요하게 동작함
- 모델이 복잡할수록 임베딩 계층이 토큰 의미 정보를 학습하기는 더 어려움

6.28 사전 학습된 모델로 임베딩 계층 초기화

- 사전 학습된 모델로 임베딩 계층을 초기화 하는 경우 넘파이 배열로 초기화할 수 있음
→ 이때 `<pad>` 토큰과 `<unk>` 토큰은 초기화에서 제외
- 임베딩 계층은 파이토치의 임베딩 클래스의 `from_pretrained` 메서드로 초기화 가능 → 이 값을 `SentenceClassifier` 클래스의 `self.embedding` 값으로 적용

6.29 사전 학습된 임베딩 계층 적용

6.30 사전 학습된 임베딩을 사용한 모델 학습

- 사전 학습된 임베딩을 사용하는 것은 모델 성능을 개선할 수 있는 방법 중 하나
but, 학습데이터의 양이 충분히 많다면, 모델의 목적에 맞게 새로운 임베딩 층을 학습하는것이 더 좋은 결과를 얻을 수 있음
- 사전 학습된 임베딩을 사용하더라도 해당 언어와 문제에 맞는 임베딩을 선택하는 것이 중요

5 합성곱 신경망

- 합성곱 신경망(CNN)
 - 주로 이미지인식과 같은 컴퓨터 비전 분야의 데이터를 분석하기 위해 사용되는 인공신경망의 한 종류
 - 입력 데이터의 지역적인 특징을 추출하는 데 특화된 구조를 갖고 있으며, 이를 위해 합성곱 연산 사용
 - 합성곱 연산 : 이미지의 특정 영역에서 입력값의 분포 또는 변화량을 계산해 출력 노드 생성, 특정 영역 안에서 연산 수행 → 지역 특징 효과적으로 추출
- 이미지 데이터 : 고정된 프레임 내에 객체들의 위치와 형태가 자유분방 → 여러 영역의 지역 특징을 조합해 입력 데이터의 전반적인 전역특징 파악 가능
- 합성곱 신경망은 원래 컴퓨터 비전을 위해 고안된 인공신경망 but, 자연어 처리 작업에서도 우수한 성능 보임
 - ↔ 순환신경망 : 이전 시점($t-1$)의 상태를 기억해 현재 상태(t)를 계산하는 데 유용한 구조지만, 연산순서의 제약으로 병렬처리가 어렵다는 단점
 - + 입력 데이터의 길이 ↑ ⇒ 처리속도 ↓ , 가중치 개수 ↑ ⇒ 학습 난이도 ↑ 문제
- 사전 학습된 임베딩 + 합성곱 신경망 이용 문장 분류 모델 등장 → 자연어 처리에서도 합성곱 신경망 사용 시작
 - ⇒ 입력 데이터의 길이에 상관없이 병렬처리 가능 + 학습에 필요한 가중치 수를 줄여 깊은 신경망 구성 가능

1. 합성곱 계층

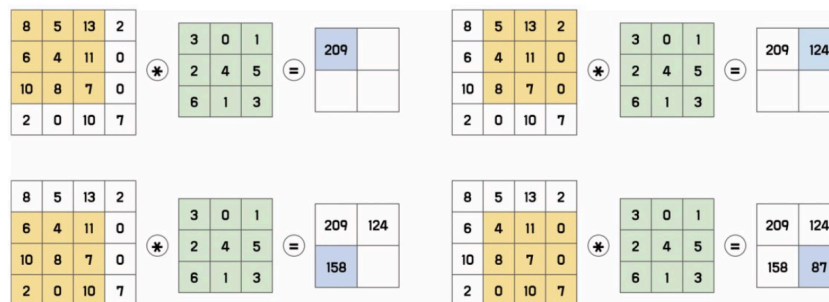
: 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층

- 이미지나 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용
- 필터 사용해 데이터의 특징을 추출 → 데이터의 지역적인 패턴 인식 가능
- 입력 데이터의 모든 위치에서 동일한 필터를 사용 → 모델 매개변수 공유

⇒ 모델이 학습해야 할 매개변수 수 ↓ = 과대적합 방지

⇒ 입력데이터에서 특징을 추출할 때, 특징이 어디에 있어도 동일하게 추출 가능

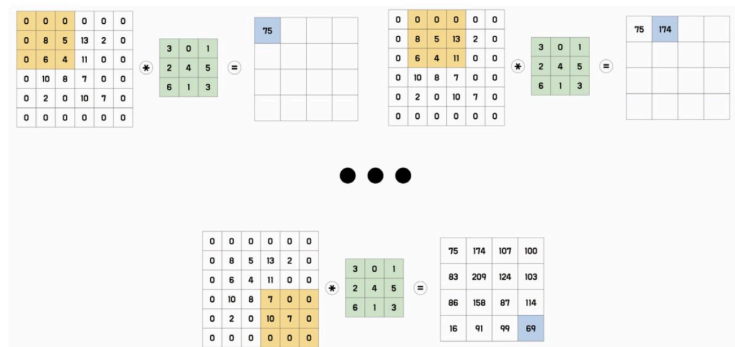
- 합성곱 계층을 여러겹 쌓아 모델 구성 → 합성곱 계층 ↑ → 모델 복잡도 ↑ → 더 다양한 특징 추출해 학습 가능
- 필터 (= 커널 / 윈도우)
 - 일반적으로 3x3, 5x5와 같은 작은 크기의 정방형으로 구성
 - 필터를 일정간격으로 이동하면서 입력 데이터와 합성곱 연산 수행 → 특징맵 생성
 - 필터 영역마다 합성곱 연산이 수행되며, 필터의 가중치가 이 모델 학습과정에서 갱신됨
 - 합성곱 신경망은 보통 여러개의 필터를 사용해 다양한 특징 추출 → 입력 이미지의 다양한 특징 인식 및 분류
 - 특징맵 추출 과정



- 합성곱 연산은 입력 이미지에서 필터와 대응하는 부분을 곱한 후, 모두 더한 값을 특징맵의 한 원소로 사용
- 필터는 입력 이미지 왼쪽 상단부터 오른쪽 하단으로 이동하면서 합성곱 연산을 반복, 이를 통해 특징맵 전체 계산
- 특징맵은 다음 합성곱 계층의 입력으로 사용되어 동일한 합성곱 연산 과정 반복
- 각 계층에서 하나의 필터가 여러 번 사용, 이를 공유 가중치로 공유함 → 이미지 내에서 어느 위치에서도 동일한 패턴 학습 가능
- 패딩
 - 입력 이미지나 입력으로 사용되는 특징맵 가장자리에 특정 값을 덧붙임
 - 일반적으로 합성곱 연산 수행 시 출력값인 특징맵의 크기가 작아짐으로써 발생하는 현상을 방지하기 위해서 패딩 추가

- 합성곱 신경망을 더 깊게 쌓는 데 제약 사항이 될 수 있음
- 합성곱 신경망 성능에 악영향 끼칠 수 있음
- 이미지의 가장자리에 있는 정보는 다른 위치에 있는 정보에 비해 학습하기 더 어려움

◦ 제로 패딩 : 가장자리에 덧붙이는 패딩 값은 0으로 할당



◦ 패딩을 적절히 사용해 입력 데이터의 크기 조정, 작은 피러로도 이미지 전에 대한 정보 반영 가능

• 간격

- 필터가 한번에 움직이는 크기
- 간격 크기 조절 → 출력 데이터 크기 조절 가능
- 간격 조정 → 입력 데이터의 공간적인 정보 유지 or 감소 가능
 - 입력 데이터의 공간적인 정보: 픽셀 간의 상대적인 위치나 거리에 대한 정보
 - 간격 ↓ : 입력 데이터의 공간적인 정보 보존 ↔ 간격 ↑ : " 감소
- 간격 조절 → 모델 매개 변수 수 ↓ → 모델 복잡도 ↓ 과대적합 방지 가능

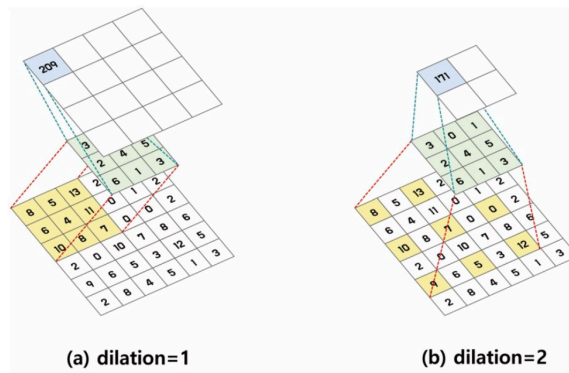
• 채널

- 입력데이터와 필터 간의 연산 수행
- 입력 데이터와 필터가 3차원으로 구성 되어있을 때 같은 위치의 값끼리 연산되도록 함
 - 입력 데이터의 공간 정보를 유지하면서 추출되는 특징 확장 가능
- ex. 입력데이터 = RGB 이미지 → 각각의 R,G,B 채널마다 동일한 필터 존재, 각 필터는 해당 채널의 정보를 추출해 특징맵 생성, 특징맵 개수 = 채널 개수
- 채널 개수 : 일반적으로 합성곱 계층에서 설정, 모델 구조/목적에 따라 달라짐

- 채널 개수 ↑ : 학습할 수 있는 특징의 다양성 ↑ 모델의 표현력 ↑
- 출력 채널 ↑ : 각 채널은 입력데이터에서 서로 다른 특징 학습 → 모델은 더 많은 종류의 특징 학습 → 더 복잡한 문제를 해결할 수 있는 능력 ↑

but, 모델 매개변수 ↑ → 학습 시간 / 메모리 사용량 ↑

- 팽창 (Dilation)



- 합성곱 연산 수행 시, 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법 = 필터와 입력 데이터 사이에 간격을 두는 방법
- 일반적으로 팽창값은 1로 사용해 간격을 한 칸만 띄워 사용, 이 값이 커질수록 필터가 입력 데이터를 바라보는 범위 ↑
- 필터의 크기 조정 없이 입력 데이터에 더 넓은 영역 고려할 수 있게 함 → 더 깊고 복잡한 모델 구성 가능 ⇒ 모델 매개변수 ↓ 메모리 사용량 ↓

↔ 필터가 바라봐야 하는 입력 데이터의 범위 ↑ : 오히려 연산량 ↑, 팽창크기가 너무 크다면 인접한 픽셀값을 고려x → 공간적인 정보 보존 x → 특징 추출 효과 ↓

- 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0,
    dilation=1,
    groups=1,
    bias=True,
    padding_mode="zeros"
)
```


- `in_channels` : 입력 데이터 채널 크기, `out_channels` : 출력 데이터 채널 크기 → 채널 생성
= 선형 변환 클래스 - `in_features` : 입력 데이터 차원 크기, `out_features` : 출력 데이터 차원 크기
- `kernel_size` : 커널 크기, 필터 크기
- `groups` : 입력 채널과 출력 채널을 하나의 그룹으로 묶는 것
→ 그룹 사용 시, 합성곱 연산 모델의 매개변수 수 감소 가능
- `bias` : 계층에 편향 값 포함 여부 설정
- `padding_mode` : 패딩 영역에 할당되는 값 설정
 - `zeros` : 제로패딩
 - `reflect` : 입력데이터의 가장자리를 거울처럼 반사해 값을 할당
 - `replicate` : 가장자리의 값을 복사해 값을 할당
- 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

2. 활성화 맵

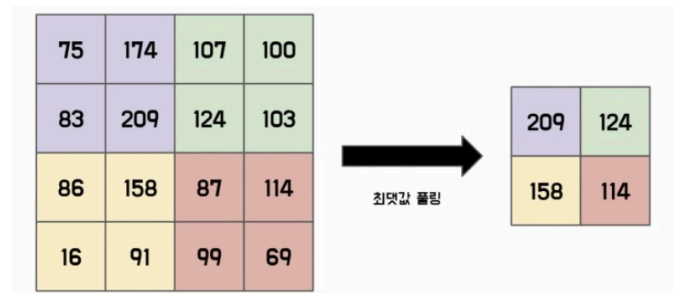
: 합성곱 계층의 특징맵에 활성화 함수를 적용해 얻어진 출력값

- 합성곱 계층에서 입력 이미지와 필터의 합성곱 연산을 통해 특징맵 추출 → 이 값에 비선형성을 추가하기 위해 활성화 함수 적용
 - 일반적으로 ReLU 함수 적용 : 특징맵의 값이 0보다 크면 그값을 그대로 출력하고, 0 이하일 경우에는 0을 출력
- 활성화 맵은 다음 계층의 입력값으로 사용
- 합성곱 연산과 활성화 함수 여러번 반복하여 신경망 구성 → 입력 이미지에서 추출된 추상적인 특징 학습 가능
↔ 합성곱 계층의 출력값에 활성화 함수 적용 X → 합성곱 연산 결과값이 그대로 다음 계층으로 전달 → 모델이 선형적인 결합만 수행 → 복잡한 패턴 / 추상적인 특징 학습 어려워짐

3. 풀링

: 특징맵의 크기를 줄이는 연산으로 합성곱 계층 다음에 적용, 연산량을 감소시키고 입력 데이터의 정보를 압축하는 효과

- 합성곱 연산과 비슷하게 필터와 간격 이용, 일정한 크기의 필터 내 특정값 선택
 - 최댓값 풀링(MaxPooling)
 - 특정 크기의 필터내 원숫값 중 가장 큰 값 선택 → 특징맵 크기 감소



- 평균값 풀링(Average Pooling)
 - 필터 내 원숫값의 평균값으로 특징맵 크기 감소
- 입력데이터의 공간적 크기 ↓ → 계산 비용 ↓, 입력 데이터의 특징 위치가 변경되더라도 인근 영역에 대한 연산 적용 → 공간적 정보 유지
- but 입력데이터의 위치 정보 일부손실 → 세밀한 위치 정보가 필요한 작업에서는 성능 저하 초래 가능
 - ⇒ 풀링을 이용해 공간적 크기를 감소하는 방법보다 연산량이 더 많더라도 합성곱 계층의 간격을 설정해 입력 데이터의 공간적인 크기를 줄이는 방법 사용
- 풀링 클래스
 - 최댓값 풀링 클래스

```
pool = torch.nn.MaxPool2d(
    kernel_size,
    stride=None,
    padding=0,
    dilation=1
)
```

- 평균값 풀링 클래스

```
pool = torch.nn.AvgPool2d(
    kernel_size,
    stride=None,
    padding=0,
    count_include_pad=True
)
```

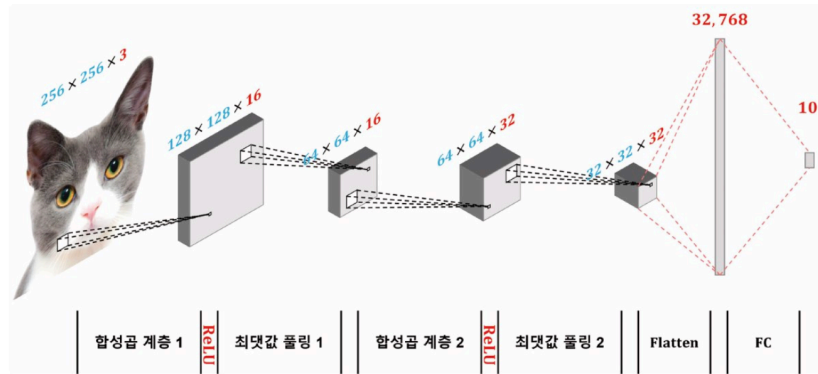
- `count_include_pad` : 패딩영역의 값을 평균 계산에 포함할지 여부 설정
- 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

4. 완전 연결 계층

: 각 입력 노드가 모든 출력 노드와 연결된 상태 → 완전 연결 계층은 입력과 출력 간의 모든 가능한 관계 학습

- 출력 노드 수 조절 가능 → 모델 복잡성과 용량 조절하는 데 사용
- 일반적으로 합성곱 신경망에서는 합성곱 계층과 풀링 계층을 거친 결과물인 특징맵 입력으로 받음 → 입력된 3차원 특징맵은 평탄화 작업을 통해 1차원 벡터로 변경 → 완전 연결 계층의 가중치와 내적연산 수행 → 출력값 계산
 - 이전 계층에서 추출한 특성 맵의 공간정보가 무시되고 모든 입력을 독립적으로 처리해 계산
- 이전 계층에서 추출된 특징을 활용하여 최종적인 분류 작업 수행
- 최종값은 소프트맥스/시그모이드와 같은 활성화함수 적용 → 분류 모델로 구성
- 완전 연결 계층 = 합성곱 신경망의 최종 출력을 결정하는 역할
- 합성곱 모델 구조 시각화

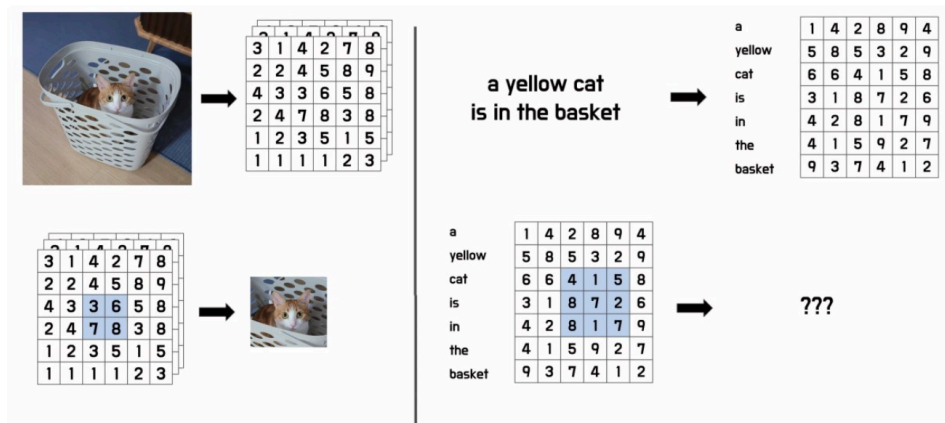


6.31 합성곱 모델

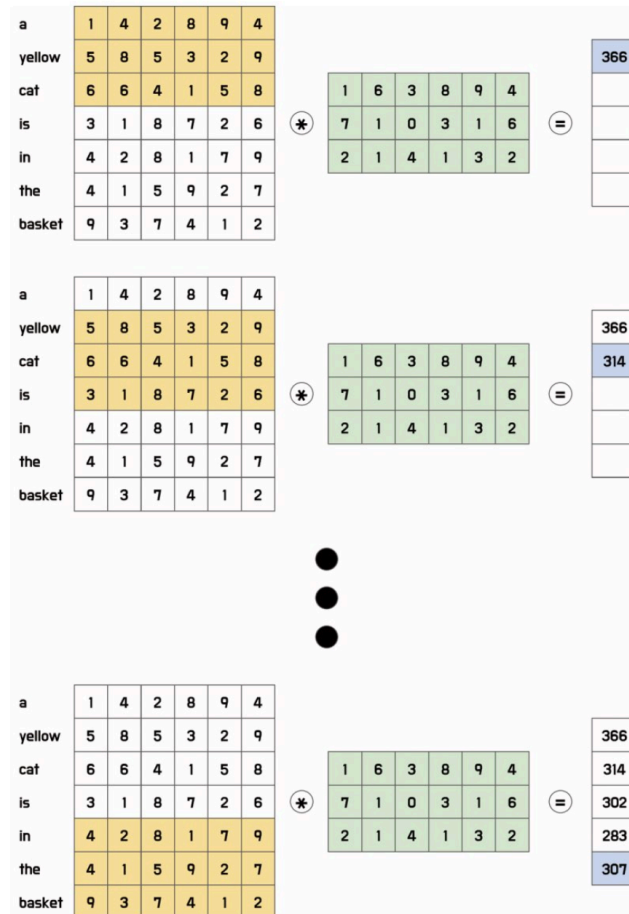
- 완전 연결 계층은 선형 변환으로 구성, 입력 벡터와 가중치 곱으로 계산
- 완전 연결 계층의 입력 데이터 차원 크기 = 마지막 특징맵 높이 x 너비 x 채널로 계산
- 출력 데이터 차원 크기 = 분류 모델의 클래스 개수

5. 모델 실습

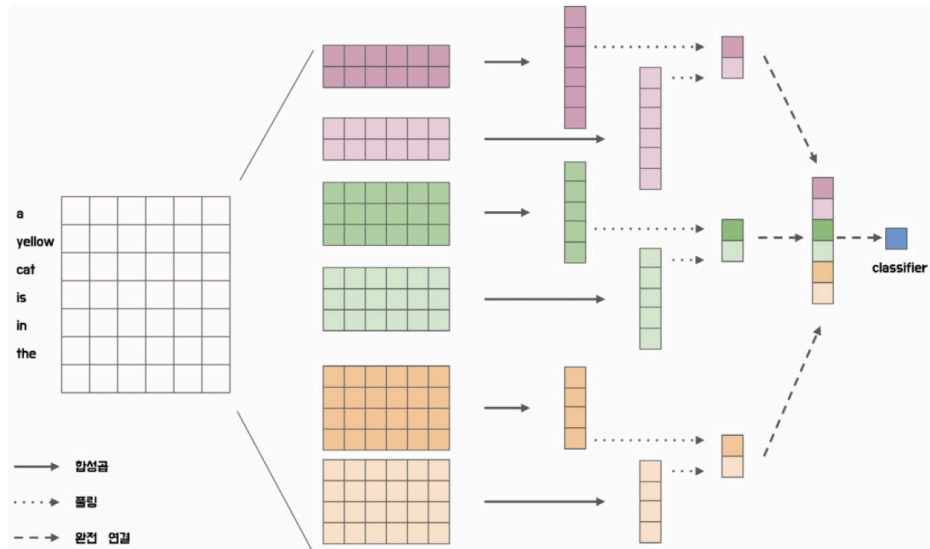
- 이미지 데이터와 텍스트 데이터의 2차원 합성곱



- 텍스트 데이터는 2차원 합성곱이 아닌, 1차원 합성곱 적용
- 1차원 합성곱
 - 일반적으로 텍스트 데이터 처리에서 많이 사용
 - 텍스트 데이터는 문장을 단어 단위로 분리하여 각 단어를 임베딩 하여 나온 1차원 벡터 데이터를 입력값으로 사용
 - 수평 방향으로 이동하지 않고 수직 방향으로만 이동하는 필터 적용하여 해당 문장의 특징 추출
 - 1차원 필터의 크기는 필터의 높이에만 영향, 필터 너비 = 입력 임베딩의 크기



- 1차원 합성곱을 이용한 신경망에서는 다양한 크기의 합성곱 필터 사용하여 여러 종류의 정보 추출 가능
- 1차원 합성곱 수행하면 1차원 벡터를 출력값으로 얻게 되므로 풀링을 적용하면 하나의 스칼라값이 도출됨
 - 크기가 다른 여러 개의 합성곱 필터를 사용 시 여러 개의 스칼라값을 얻을 수 있음 → 다양한 크기의 출력 벡터를 모아 하나의 벡터로 연결하여 하나의 특징 벡터로 만들 수 있음



6.32 합성곱 기반 문장 분류 모델 정의

- `SentenceClassifier` 클래스 : 사전 학습된 임베딩 벡터 `pretrained_embedding`, 필터 크기 리스트 `filter_sizes`, 최대길이 `max_length` 입력 받아 모델 구성
 - 합성곱 계층 구성 시 크기가 다른 여러 개의 합성곱 필터를 구성하기 위해 필터 크기 리스트 만큼 시퀀셜 생성
 - 각 시퀀스는 1차원 ~~합성곱~~ 합성곱 계층, ReLU, 최댓값 풀링으로 구성
- `ModuleList` 클래스 : 여러 개의 서브 모듈을 리스트 형태로 저장하는 기능 제공, conv리스트에는 시퀀스 모듈이 담겨있으므로 `ModuleList` 클래스 적용
- 순방향 메서드 : 입력값을 임베딩 계층에 통과시켜 임베딩 벡터 얻음 → `permute` 메서드를 통해 차원을 변경하고 앞서 설정한 여러 개의 합성곱 필터에 통과시킴
- `cat` 메서드 : 각 필터의 출력값을 이어 붙여 하나의 벡터로 만들고 분류를 위한 네트워크를 통과시켜 최종 출력값 얻음

6.33 합성곱 신경망 분류 모델 학습

- `Adam` : 경사 하강법 알고리즘을 개선한 방법, 모멘텀과 RMSProp을 결합한 방법
 - = 모멘텀 : 지난 기울기 값의 영향을 일정 부분 유지하면서 새로운 기울기 값을 반영하기 → 기울기가 빠르게 변화하는 구간에서 더 안정적인 수렴 보임
 - = RMSProp: 학습률을 조절해 발산하는 경우 방지
 - 학습률, 모멘텀, RMSProp의 하이퍼파라미터를 모두 자동으로 조절하면서 최적의 값으로 수렴 → 하이퍼파라미터 조정하는 수고 ↓, 학습률과 모멘텀의 조절 문제 해결 →여 SGD보다 더 빠른 수렴을 보이는 경우 多